# Let's get started

We want you to be successful.

We will work together to build an environment in CSE 020 that supports your learning in a way that respects your perspectives, experiences, and identities. Our goal is for you to engage with interesting and challenging concepts and feel comfortable exploring, asking questions, and thriving.

If you or someone you know is suffering from food and/or housing insecurities there are UCSD resources here to help:

Basic Needs Office: https://basicneeds.ucsd.edu/

Triton Food Pantry (in the old Student Center) is free and anonymous, and includes produce:

https://www.facebook.com/tritonfoodpantry/

Financial aid resources, the possibility of emergency grant funding, and off-campus housing referral resources are available: see your College Dean of Student Affairs.

If you find yourself in an uncomfortable situation, ask for help. We are committed to upholding University policies regarding nondiscrimination, sexual violence and sexual harassment. Here are some campus contacts that could provide this help: Counseling and Psychological Services (CAPS) at 858 534-3755 or http://caps.ucsd.edu; OPHD at 858 534-8298 or ophd@ucsd.edu , http://ophd.ucsd.edu; CARE at Sexual Assault Resource Center at 858 534-5793 or sarc@ucsd.edu , http://care.ucsd.edu.

Please reach out (minnes@ucsd.edu) if you need support with extenuating circumstances affecting CSE 020.

CC BY-NC-SA 2.0 Version October 27, 2026 (1)

# Welcome to CSE 020: Discrete Math for CS in Fall 2026!

Class website: https://canvas.ucsd.edu/ Instructor: Prof. Mia Minnes "Minnes" rhymes with Guinness, minnes@ucsd.edu, http://cseweb.ucsd.edu/ minnes. CSE 020 team: One instructor + two TAs and three tutors + all of you

Fill in contact info for students around you, if you'd like:

What you can find on the class website on Canvas (https://canvas.ucsd.edu/):

1. Syllabus
2. Notes for class + annotations
3. PrairieLearn
4. Assignments (PDF, tex, solutions)
5. Gradescope
6. Piazza
7. Course calendar for HW due dates and Q&A, tests, etc.

There are lots of great reasons to have a laptop, tablet, or phone open during class. You might be taking notes, getting a photo of an important moment on the board, trying out a construction that we're developing together, working on the review quiz, and so on. The main issue with screens and technology in the classroom isn't that they might distract you, it's the distraction of other students. We ask that if you would like to use a device in class and may have unrelated content open, please sit in one of the back two rows of the room so that it's not adversely affecting other students.

**Pro-tip**: you can use MATH 109 to replace CSE 020 for prerequisites and other requirements.

# Themes and applications for CSE 020

- **Technical skepticism**: Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.
- **Multiple representations**: Understand, guide, shape impact of computing on society/the world. Connect the role of Theory CS classes to other applications (in undergraduate CS curriculum and beyond). Model problems using appropriate mathematical concepts. Clearly and unambiguously communicate computational ideas using appropriate formalism. Translate across levels of abstraction.

**Applications**: Numbers (how to represent them and use them in Computer Science), Recommendation systems and their roots in machine learning (with applications like Netflix), "Under the hood" of computers (circuits, pixel color representation, data structures), Codes and information (secret message sharing and error correction), Bioinformatics algorithms and genomics (DNA and RNA).

CC BY-NC-SA 2.0 Version October 27, 2026 (2)

## Week 1 at a glance

**We will be learning and practicing to:**

- Model systems with tools from discrete mathematics and reason about implications of modelling choices. Explore applications in CS through multiple perspectives, including software, hardware, and theory.
  - Selecting and representing appropriate data types and using notation conventions to clearly communicate choices
- Translate between different representations to illustrate a concept.
  - Translating between symbolic and English versions of statements using precise mathematical language
  - Tracing algorithms specified in pseudocode
- Use precise notation to encode meaning and present arguments concisely and clearly
  - Defining important sets of numbers, e.g. set of integers, set of rational numbers
  - Precisely describing a set using appropriate notation e.g. roster method, set builder notation, and recursive definitions
  - Defining functions using multiple representations
- Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.
  - Using a recursive definition to evaluate a function or determine membership in a set
  - Using the definitions of the div and mod operators on integers

**TODO:**

#FinAid Assignment on Canvas `https://canvas.ucsd.edu/courses/77744/assignments/1192300`

Read the syllabus on Canvas

Schedule your Test 1 and Test 2 times at PrairieTest (http://us.prairietest.com)

Review quiz based on class material (due Monday of Week 2)

Create a homework group, possibly by using the Piazza (https://piazza.com/ ) find-a-teammate tool

Homework assignment 1 (due Tuesday of Week 2)

CC BY-NC-SA 2.0 Version October 27, 2026 (3)

# Week 0 Friday: Modeling applications

What data should we encode about each Netflix account holder to help us make effective recommendations?

In machine learning, clustering can be used to group similar data for prediction and recommendation. For example, each Netflix user's viewing history can be represented as a $n$-tuple indicating their preferences about movies in the database, where $n$ is the number of movies in the database. People with similar tastes in movies can then be clustered to provide recommendations of movies for one another. Mathematically, clustering is based on a notion of distance between pairs of $n$-tuples.

## Data Types: sets, $n$-tuples, and strings

| **Term** | **Examples**:<br>(add additional examples from class) |
|---|---|
| **set**<br>unordered collection of elements<br>*repetition doesn't matter*<br>*Equal sets agree on membership of all elements* | $7 \in \{43, 7, 9\}$ $\quad 2 \notin \{43, 7, 9\}$ |
| **$n$-tuple**<br>ordered sequence of elements with $n$ "slots" $(n > 0)$<br>*repetition matters, fixed length*<br>*Equal $n$-tuples have corresponding components equal* | |
| **string**<br>ordered finite sequence of elements each from specified set (called the alphabet over which the string is defined)<br>*repetition matters, arbitrary finite length*<br>*Equal strings have same length and corresponding characters equal* | |

*Special cases*:

When $n = 2$, the 2-tuple is called an **ordered pair**.

A string of length 0 is called the **empty string** and is denoted $\lambda$.

A set with no elements is called the **empty set** and is denoted $\{\}$ or $\emptyset$.

CC BY-NC-SA 2.0 Version October 27, 2026 (4)

In the table below, each row represents a user's ratings of movies: ✓ (check) indicates the person liked the movie, ✗ (x) that they didn't, and • (dot) that they didn't rate it one way or another (neutral rating or didn't watch). Can encode these ratings numerically with 1 for ✓ (check), −1 for ✗ (x), and 0 for • (dot).

| Person | Superman | Wicked | KPop Demon Hunters | A Minecraft Movie | Ratings written as a 4-tuple |
|---|---|---|---|---|---|
| $P_1$ | ✗ | • | ✓ | | |
| $P_2$ | ✓ | ✓ | ✗ | | |
| $P_3$ | ✓ | ✓ | ✓ | | |
| $P_4$ | • | ✗ | ✓ | | |
| $You$ | | | | | |

**Conclusion**: Modeling involves choosing data types to represent and organize data

CC BY-NC-SA 2.0 Version October 27, 2026 (5)

# Week 1 Monday: Defining sets

## Notation and prerequisites

| Term | Notation Example(s) | We say in English ... |
|---|---|---|
| all reals | $\mathbb{R}$ | The (set of all) real numbers (numbers on the number line) |
| all integers | $\mathbb{Z}$ | The (set of all) integers (whole numbers including negatives, zero, and positives) |
| all positive integers | $\mathbb{Z}^+$ | The (set of all) strictly positive integers |
| all natural numbers | $\mathbb{N}$ | The (set of all) natural numbers (nonnegative integers). *We use the convention that* 0 *is a natural number.* |

*To define sets:*

To define a set using **roster method**, explicitly list its elements. That is, start with { then list elements of the set separated by commas and close with }.

To define a set using **set builder definition**, either form "The set of all $x$ from the universe $U$ such that $x$ is ..." by writing

$$\{x \in U \mid ...x...\}$$

or form "the collection of all outputs of some operation when the input ranges over the universe $U$" by writing

$$\{...x... \mid x \in U\}$$

We use the symbol $\in$ as "is an element of" to indicate membership in a set.

**Example sets**: For each of the following, identify whether it's defined using the roster method or set builder notation and give an example element.

Can we infer the data type of the example element from the notation?

$\{-1, 1\}$

$\{0, 0\}$

$\{-1, 0, 1\}$

$\{(x, x, x) \mid x \in \{-1, 0, 1\}\}$

$\{\}$

$\{x \in \mathbb{Z} \mid x \geq 0\}$

$\{x \in \mathbb{Z} \mid x > 0\}$

$\{\smile, \sun\}$

$\{\texttt{A}, \texttt{C}, \texttt{U}, \texttt{G}\}$

$\{\texttt{AUG}, \texttt{UAG}, \texttt{UGA}, \texttt{UAA}\}$

RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\mathtt{A}, \mathtt{C}, \mathtt{U}, \mathtt{G}\}$. Strands are ordered nonempty finite sequences of bases.

Formally, to define the set of all RNA strands, we need more than roster method or set builder descriptions.

**New! Recursive Definitions of Sets**: The set $S$ (pick a name) is defined by:

| | |
|---|---|
| Basis Step: | Specify finitely many elements of $S$ |
| Recursive Step: | Give rule(s) for creating a new element of $S$ from known values existing in $S$, and potentially other values. |

The set $S$ then consists of all and only elements that are put in $S$ by finitely many (a nonnegative integer number) of applications of the recursive step after the basis step.

**Definition** The set of nonnegative integers $\mathbb{N}$ is defined (recursively) by:

Basis Step:
Recursive Step:

Examples:

**Definition** The set of all integers $\mathbb{Z}$ is defined (recursively) by:

Basis Step:
Recursive Step:

Examples:

**Definition** The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

Examples:

**Definition** The set of bitstrings (strings of 0s and 1s) is defined (recursively) by:

Basis Step:
Recursive Step:

*Notation:* We call the set of bitstrings $\{0,1\}^*$ and we say this is the set of all strings over $\{0,1\}$.

Examples:

CC BY-NC-SA 2.0 Version October 27, 2026 

To define a set we can use the roster method, set builder notation, a recursive definition, and also we can apply a set operation to other sets.

**New! Cartesian product of sets** and **set-wise concatenation of sets of strings**

**Definition**: Let $X$ and $Y$ be sets. The **Cartesian product** of $X$ and $Y$, denoted $X \times Y$, is the set of all ordered pairs $(x, y)$ where $x \in X$ and $y \in Y$

$$X \times Y = \{(x, y) \mid x \in X \text{ and } y \in Y\}$$

Conventions: (1) Cartesian products can be chained together to result in sets of $n$-tuples and (2) When we form the Cartesian product of a set with itself $X \times X$ we can denote that set as $X^2$, or $X^n$ for the Cartesian product of a set with itself $n$ times for a positive integer $n$.

**Definition**: Let $X$ and $Y$ be sets of strings over the same alphabet. The **set-wise concatenation** of $X$ and $Y$, denoted $X \circ Y$, is the set of all results of string concatenation $xy$ where $x \in X$ and $y \in Y$

$$X \circ Y = \{xy \mid x \in X \text{ and } y \in Y\}$$

**Pro-tip**: the meaning of writing one element next to another like $xy$ depends on the data-types of $x$ and $y$. When $x$ and $y$ are strings, the convention is that $xy$ is the result of string concatenation. When $x$ and $y$ are numbers, the convention is that $xy$ is the result of multiplication. This is (one of the many reasons) why is it very important to declare the data-type of variables before we use them.

*Fill in the missing entries in the table*:

| **Set** | **Example elements in this set and their data type**: |
|---|---|
| $B$ | `A` `C` `G` `U` |
| | $(\texttt{A}, \texttt{C})$ $(\texttt{U}, \texttt{U})$ |
| $B \times \{-1, 0, 1\}$ | |
| $\{-1, 0, 1\} \times B$ | |
| | $(0, 0, 0)$ |
| $\{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{U}\} \circ \{\texttt{A}, \texttt{C}, \texttt{G}, \texttt{U}\}$ | |
| | `GGGG` |

# Week 1 Wednesday: Defining functions

| Term | Notation Example(s) | We say in English ... |
|---|---|---|
| sequence | $x_1, \ldots, x_n$ | A sequence $x_1$ to $x_n$ |
| summation | $\sum_{i=1}^{n} x_i$ or $\sum_{i=1}^{n} x_i$ | The sum of the terms of the sequence $x_1$ to $x_n$ |
| piecewise rule definition | $f(x) = \begin{cases} \text{rule 1 for } x & \text{when COND 1} \\ \text{rule 2 for } x & \text{when COND 2} \end{cases}$ | Define $f$ of $x$ to be the result of applying rule 1 to $x$ when condition COND 1 is true and the result of applying rule 2 to $x$ when condition COND 2 is true. This can be generalized to having more than two conditions (or cases). |
| function application | $f(7)$ | $f$ of 7 **or** $f$ applied to 7 **or** the image of 7 under $f$ |
| | $f(z)$ | $f$ of $z$ **or** $f$ applied to $z$ **or** the image of $z$ under $f$ |
| | $f(g(z))$ | $f$ of $g$ of $z$ **or** $f$ applied to the result of $g$ applied to $z$ |
| absolute value | $\lvert -3 \rvert$ | The absolute value of $-3$ |
| square root | $\sqrt{9}$ | The non-negative square root of 9 |

**Pro-tip**: the meaning of two vertical lines | | depends on the data-types of what's between the lines. For example, when placed around a number, the two vertical lines represent absolute value. We've seen a single vertial line | used as part of set builder definitions to represent "such that". Again, this is (one of the many reasons) why is it very important to declare the data-type of variables before we use them.

**New! Defining functions** A function is defined by its (1) domain, (2) codomain, and (3) rule assigning each element in the domain exactly one element in the codomain.

The domain and codomain are nonempty sets.
The rule can be depicted as a table, formula, piecewise definition, or English description.
The notation is

"Let the function FUNCTION-NAME: DOMAIN $\to$ CODOMAIN be given by
FUNCTION-NAME(x) = ... for every $x \in DOMAIN$".

or

"Consider the function FUNCTION-NAME: DOMAIN $\to$ CODOMAIN defined as
FUNCTION-NAME(x) = ... for every $x \in DOMAIN$".

CC BY-NC-SA 2.0 Version October 27, 2026 (10)

Example: The absolute value function

**Domain**

**Codomain**

**Rule**

Recall our representation of Netflix users' ratings of movies as $n$-tuples, where $n$ is the number of movies in the database. Each component of the $n$-tuple is $-1$ (didn't like the movie), 0 (neutral rating or didn't watch the movie), or 1 (liked the movie).

Consider the ratings $P_1 = (-1, 0, 1, 0)$, $P_2 = (1, 1, -1, 0)$, $P_3 = (1, 1, 1, 0)$, $P_4 = (0, -1, 1, 0)$

Which of $P_1$, $P_2$, $P_3$ has movie preferences most similar to $P_4$?

One approach to answer this question: use **functions** to quantify difference among user preferences.

For example, consider the function $d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \to \mathbb{R}$ given by

$$d_0(\ (\ (x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)\ )\ ) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

CC BY-NC-SA 2.0 Version October 27, 2026 (11)

When the domain of a function is a *recursively defined set*, the rule assigning images to domain elements (outputs) can also be defined recursively.

Recall: The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

**Definition** (Of a function, recursively) A function *rnalen* that computes the length of RNA strands in $S$ is defined by:

| | | | |
|---|---|---|---|
| | | $rnalen : S$ | $\to \mathbb{Z}^+$ |
| Basis Step: | If $b \in B$ then | $rnalen(b)$ | $= 1$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then | $rnalen(sb)$ | $= 1 + rnalen(s)$ |

The domain of *rnalen* is

The codomain of *rnalen* is

Example function application:

$$rnalen(\mathtt{ACU}) =$$

*Example*: A function *basecount* that computes the number of a given base $b$ appearing in a RNA strand $s$ is defined recursively:

| | | | | |
|---|---|---|---|---|
| | | $basecount : S \times B$ | $\to \mathbb{N}$ | |
| Basis Step: | If $b_1 \in B, b_2 \in B$ | $basecount(\ (b_1, b_2)\ )$ | $= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$ | |
| Recursive Step: | If $s \in S, b_1 \in B, b_2 \in B$ | $basecount(\ (sb_1, b_2)\ )$ | $= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | |

CC BY-NC-SA 2.0 Version October 27, 2026 (12)

# Week 1 Friday: Sets, functions, and algorithms

Let's practice with functions related to some of our applications so far.

Recall: We model the collection of user ratings of the four movies Superman, Wicked, KPop Demon Hunters, A Minecraft Movie as the set $\{-1, 0, 1\}^4$ . One function that compares pairs of ratings is

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \to \mathbb{R}$$

given by

$$d_0(\ (\ (x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)\ )\ ) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Notice: any ordered pair of ratings is an okay input to $d_0$.

Notice: there are (at most)

$$(3 \cdot 3 \cdot 3 \cdot 3) \cdot (3 \cdot 3 \cdot 3 \cdot 3) = 3^8 = 6561$$

many pairs of ratings. There are therefore lots and lots of real numbers that are not the output of $d_0$.

Recall: RNA is made up of strands of four different bases that encode genomic information in specific ways. The bases are elements of the set $B = \{\texttt{A}, \texttt{C}, \texttt{U}, \texttt{G}\}$. The set of RNA strands $S$ is defined (recursively) by:

| | |
|---|---|
| Basis Step: | $\texttt{A} \in S, \texttt{C} \in S, \texttt{U} \in S, \texttt{G} \in S$ |
| Recursive Step: | If $s \in S$ and $b \in B$, then $sb \in S$ |

where $sb$ is string concatenation.

**Pro-tip**: informal definitions sometime use $\cdots$ to indicate "continue the pattern". Often, to make this pattern precise we use recursive definitions.

CC BY-NC-SA 2.0 Version October 27, 2026 (13)

| Name | Domain | Codomain | Rule | Example |
|---|---|---|---|---|
| *rnalen* | $S$ | $\mathbb{Z}^+$ | Basis Step:<br>If $b \in B$ then $rnalen(b) = 1$<br>Recursive Step:<br>If $s \in S$ and $b \in B$, then<br>$rnalen(sb) = 1 + rnalen(s)$ | $rnalen(\texttt{AC}) \overset{\text{rec step}}{=} 1 + rnalen(\texttt{A})$<br>$\overset{\text{basis step}}{=} 1 + 1 = 2$ |
| *basecount* | $S \times B$ | $\mathbb{N}$ | Basis Step:<br>If $b_1 \in B, b_2 \in B$ then<br>$basecount(\ (b_1, b_2)\ ) =$<br>$\begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$<br>Recursive Step:<br>If $s \in S, b_1 \in B, b_2 \in B$<br>$basecount(\ (sb_1, b_2)\ ) =$<br>$\begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$ | $basecount(\ (\texttt{ACU}, \texttt{C})\ ) =$ |
| "2 to the power of" | $\mathbb{N}$ | $\mathbb{N}$ | Basis Step:<br>$2^0 = 1$<br>Recursive Step:<br>If $n \in \mathbb{N}, 2^{n+1} =$ | |
| "$b$ to the power of $i$" | $\mathbb{Z}^+ \times \mathbb{N}$ | $\mathbb{N}$ | Basis Step:<br>$b^0 = 1$<br>Recursive Step:<br>If $i \in \mathbb{N}, b^{i+1} = b \cdot b^i$ | |

$2^0 = 1$ $2^1 = 2$ $2^2 = 4$ $2^3 = 8$ $2^4 = 16$ $2^5 = 32$ $2^6 = 64$ $2^7 = 128$ $2^8 = 256$ $2^9 = 512$ $2^{10} = 1024$

CC BY-NC-SA 2.0 Version October 27, 2026 (14)

**Integer division and remainders** (aka The Division Algorithm) Let $n$ be an integer and $d$ a positive integer. There are unique integers $q$ and $r$, with $0 \leq r < d$, such that $n = dq + r$. In this case, $d$ is called the divisor, $n$ is called the dividend, $q$ is called the quotient, and $r$ is called the remainder.

Because these numbers are guaranteed to exist, the following functions are well-defined:

**div** $: \mathbb{Z} \times \mathbb{Z}^+ \to \mathbb{Z}$ given by **div** $(\ (n, d)\ )$ is the quotient when $n$ is the dividend and $d$ is the divisor.

**mod** $: \mathbb{Z} \times \mathbb{Z}^+ \to \mathbb{Z}$ given by **mod** $(\ (n, d)\ )$ is the remainder when $n$ is the dividend and $d$ is the divisor.

Because these functions are so important, we sometimes use the notation $n$ **div** $d =$ **div** $(\ (n, d)\ )$ and $n$ **mod** $d =$ **mod** $(\ (n, d)\ )$.

**Pro-tip**: The functions **div** and **mod** are similar to (but not exactly the same as) the operators / and % in Java and python.

*Example calculations*:

20 **div** 4

20 **mod** 4

20 **div** 3

20 **mod** 3

$-20$ **div** 3

$-20$ **mod** 3

CC BY-NC-SA 2.0 Version October 27, 2026 (15)